

Beyond the Simple Queue

Deque(**Double-Ended**), Priority Queue, and the Linked Queue

Unit II • Lecture 14

Part A Deque - both ends open

Part B Priority Queue - importance, not order

Part C Linked Queue - no array limit

Recap: what a plain queue insists on

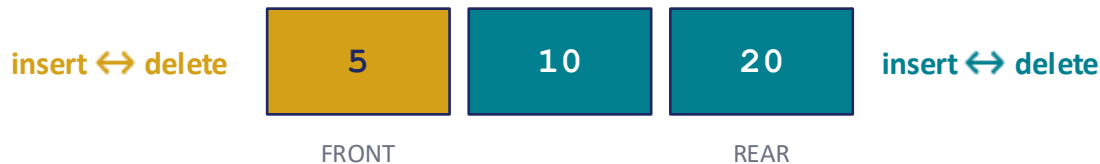
The plain queue's two rules

Rule 1: insertion ONLY at rear.

Rule 2: deletion ONLY at front. And every element leaves in the order it arrived.

- Today we relax Rule 1+2 (deque: both ends do both jobs).
- Then we drop the FIFO promise entirely (priority queue: importance decides, not arrival time).
- Then we rebuild the plain queue on linked nodes, exactly as Lecture 9 did for the stack.

Part A: the Deque (Double-Ended Queue)



- "Deque" = Double-Ended QUEUE. Say it as "deck".
- BOTH ends allow BOTH insertion and deletion — the queue's one-way rule is gone.
- Four operations: insertFront, insertRear, deleteFront, deleteRear.

Deque: array version, both ends move

```
void insertFront(int x)
{
    if (isFull()) { ...; return; }
    if (front == -1)
        front = rear = MAX / 2;
    else if (front == 0)
        { ...; return; } /* no room left */
    else
        front = front - 1;
    dq[front] = x;
}
```

- insertRear is the mirror image: rear = rear + 1.
- deleteFront and deleteRear mirror push/pop from either end.
- New trick: the FIRST element is placed in the MIDDLE of the array, not at index 0 — that leaves room to grow both left and right.

This simple version still has a limit: it is not circular, so it can still run out of room on one side while the other has space.

Deque trace (verified)

Operation	Deque (front..rear)	front	rear
<code>insertRear(10)</code>	10	2	2
<code>insertRear(20)</code>	10 20	2	3
<code>insertFront(5)</code>	5 10 20	1	3
<code>deleteFront → 5</code>	10 20	2	3
<code>deleteRear → 20</code>	10	2	2

MAX = 5, first insert lands at index 2 (the middle) — exactly as the code intends.

Part B: the Priority Queue

The rule changes completely

Forget arrival order. Every element carries a PRIORITY. The element removed next is always the one with the HIGHEST priority currently waiting.

- Emergency room: critical patients are seen before those who arrived earlier but are stable.
- Print spooler: an admin's urgent print job can jump ahead of a large queued report.
- CPU scheduling: high-priority system tasks run before ordinary background ones.

Priority Queue: the simplest working version

```
int pq[MAX];      /* values      */
int prio[MAX];    /* priorities */
int count = 0;

void insert(int value, int priority)
{
    if (isFull()) { ...; return; }
    pq[count] = value;
    prio[count] = priority;
    count++;
}
```

- insert is $O(1)$ — just append, no ordering work yet.
- removeHighest does the work instead: scan all elements, remember the index of the largest priority.
- This is the simplest correct design, not the fastest — more on that below.

Bigger number = higher priority, in this version.

removeHighest(): find, take, close the gap

```
int removeHighest(void)
{
    if (isEmpty()) { ...; return -1; }

    int hi = 0;
    for (int i = 1; i < count; i++)
        if (prio[i] > prio[hi])
            hi = i;

    int value = pq[hi];

    for (int i = hi; i < count-1; i++)
    {
        pq[i]    = pq[i+1];
        prio[i] = prio[i+1];
    }
    count--;
    return value;
}
```

Insert order (value:priority)

10:2 20:5 30:1 40:4

removeHighest called 4 times

20 → 40 → 10 → 30
(priorities 5, 4, 2, 1 — highest first)

Note on efficiency

Today's version

insert: $O(1)$ removeHighest: $O(n)$ (must scan everything to find the max)

Fine for small, teaching-scale queues.

The real answer, later in this course

A HEAP (Unit III) gives both insert and removeHighest in $O(\log n)$ — the standard, efficient priority queue used in real systems.

Part C: the Linked Queue

Same move as Lecture 9, applied to the queue

Replace the fixed array with malloc'd nodes — no MAX, no overflow (until the heap itself is full), grows exactly as needed.

- The stack needed ONE pointer (top). The queue needs TWO: front and rear.
- front lets us dequeue quickly; rear lets us enqueue quickly — without either, one operation would need a slow $O(n)$ walk.
- No circular trick needed at all — nodes are created and freed, never reused by index.

The two structs: same idea as Lecture 9

```
typedef struct Node {  
    int      value;  
    struct Node *next;  
} Node;  
  
typedef struct {  
    Node *front;  
    Node *rear;  
} Queue;
```

- Node: identical to the linked stack's node — value plus next.
- Queue: the only real change from Lecture 9's Stack — TWO pointers instead of one.
- Empty means both are NULL together — they must always agree.

qInIt sets both to NULL, exactly like stackInIt set top to NULL.

qEnqueue() — always attach at rear

```
void qEnqueue(Queue *q, int x)
{
    Node *node = malloc(sizeof *node);
    if (node == NULL) { ...; return; }
    node->value = x;
    node->next = NULL;

    if (q->rear == NULL)
        q->front = node;    /* queue was empty */
    else
        q->rear->next = node;

    q->rear = node;        /* always true */
}
```

- Create the node exactly as the stack did.
- If the queue was empty, the new node is also the front.
- Otherwise, the OLD rear must point at it — that link is what makes the chain continuous.
- The new node becomes the rear either way — the one line outside the if.

qDequeue() — always serve from front

```
int qDequeue(Queue *q, int *out)
{
    if (qIsEmpty(q)) { ...; return 0; }

    Node *node = q->front;
    *out = node->value;
    q->front = node->next;

    if (q->front == NULL)
        q->rear = NULL;
    /* just removed the LAST node */

    free(node);
    return 1;
}
```

- Save, read, unlink, free — the exact rhythm from linked-stack pop.
- The new special case: if front becomes NULL, rear must ALSO become NULL — otherwise rear would point at a freed node.
- This is the two-pointer version of the stack's single top = NULL reset.

All three, side by side with the plain queue

	Plain Queue	Deque	Priority Queue	Linked Queue
Insert at	rear only	front or rear	anywhere (unordered)	rear only
Delete at	front only	front or rear	highest priority	front only
Order out	FIFO	either end	by priority	FIFO
Size limit	MAX (array)	MAX (array)	MAX (array)	heap memory

Same course pattern each time: define the ADT's rule, then pick a representation (array or linked) to store it.

Three relatives of the queue

- Deque: both ends allow both insertion and deletion; it can behave like a stack or a queue depending on which two operations you use.
- Priority queue: FIFO is replaced by importance — simplest version is $O(1)$ insert, $O(n)$ removeHighest; heaps (Unit III) do better.
- Linked queue: front and rear pointers replace the array — enqueue always moves rear, dequeue always moves front, and both must reset together when the queue empties.
- All three keep the same course habit: define the rule (ADT) first, choose the representation second.

EXIT TICKET Write `qEnqueue` and `qDequeue` for a linked queue that also keeps a size counter, updated correctly in both functions.